TECHNICAL ELECTIVE 4
DELIVERABLE 1

WEB-BASED SCHOOL VISITOR LOG MANAGEMENT SYSTEM

Requirements, System Design, and Technology Familiarization

Prepared by:
RENATO P. MADAYAN JR.
ALZHAQUER T. SOLOG
ARREANNE R. NACUA
ESNAIDAH B. TABON
ASRAP A. GUINDO
JUNAINA M. ETO

Submitted to:
SIR WARZIV BACARAT

Date Submitted:
November 6, 2026

PART I. TECHNOLOGY FAMILIARIZATION

1. Technology Stack Overview

The project will use Python and Django for server-side development, HTML5 and CSS3 for page structure and styling, Bootstrap 5 for responsive interface components, MySQL for persistent data storage, and Git/GitHub for version control and source-code management.

Technology	Purpose in the Project
Python	Primary programming language used by Django to implement the application's server-side logic.
Django	Python web framework used to organize the project, define URLs, views, models, templates, authentication, and database operations.
HTML5	Defines the structure and content of pages such as forms, tables, navigation, and dashboard sections.
CSS3	Controls the visual appearance, spacing, typography, and layout of the web application.
Bootstrap 5	Provides responsive grid layouts and ready-to-use components such as navbars, cards, forms, buttons, alerts, and tables.
MySQL	Relational database used to store users, visitors, school offices, and visit records.
Git	Version-control system used to track source-code changes and maintain development history.
GitHub	Online repository platform used to store the project repository and collaborate on source code.

2. Development Environment

The following table should be completed using the exact versions installed on the group's development computer. Visual Studio Code will be used as the code editor/development environment for the Django project.

Tool / Software	Version	Purpose
Visual Studio Code	1.145.0	Code editor / development environment
Python	Python 3.13.7	Programming language
Django	6.1.2	Web framework
MySQL / SQLite	3.50.4	Database management system
Bootstrap	5	Responsive frontend framework
Git	git version 2.49.0.windows.1	Version control
GitHub	Web service	Remote source-code repository
Google Chrome / Microsoft Edge	Version 157.0.8103.27 (Official Build) (64-bit)	Application testing

3. Application Screenshots

Insert screenshots of the practice Django application required during technology familiarization. Each screenshot should include a short caption explaining what is being demonstrated.

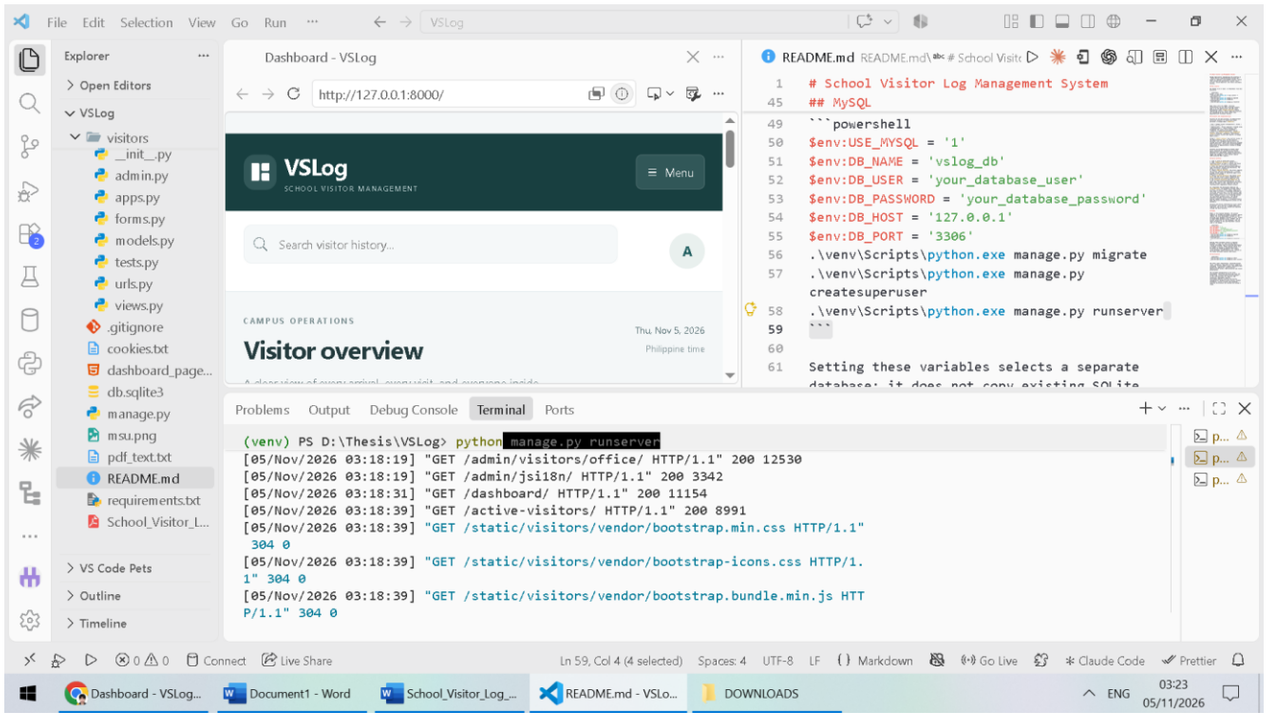


Figure 1. Practice Django application running locally.

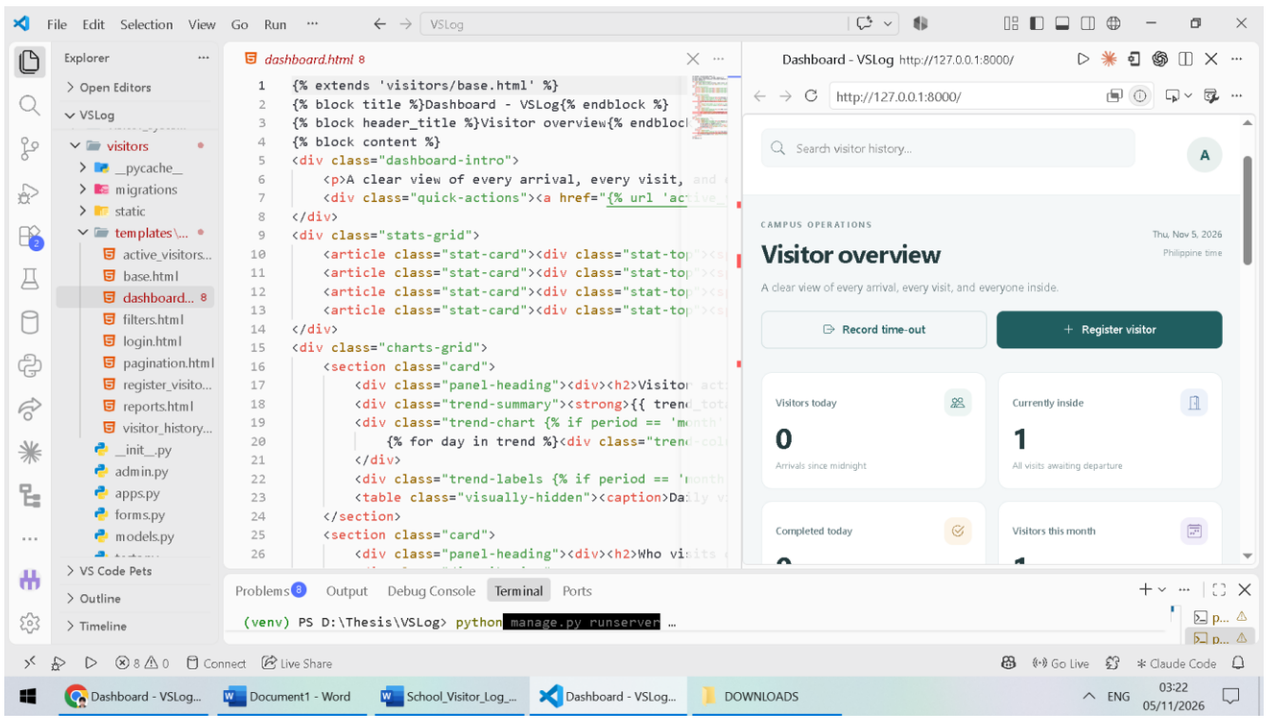


Figure 2. Sample page demonstrating Django template rendering.

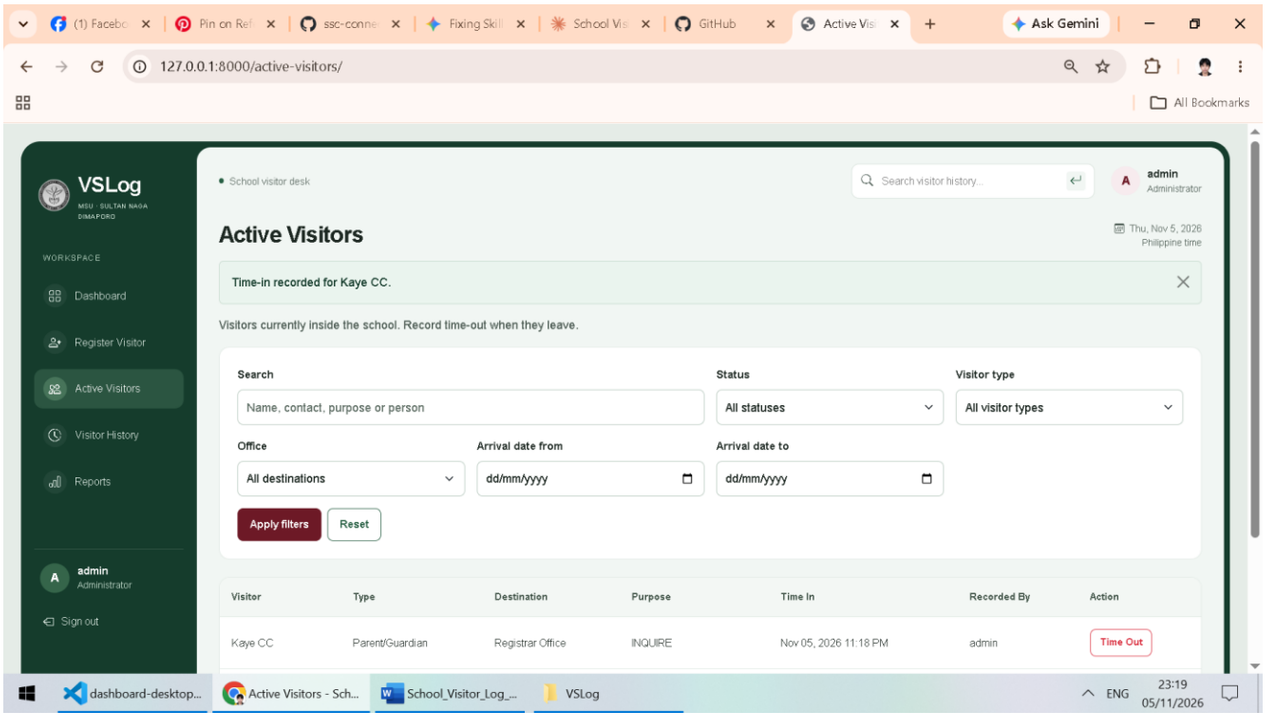


Figure 3. Sample create/read/update/delete operation.

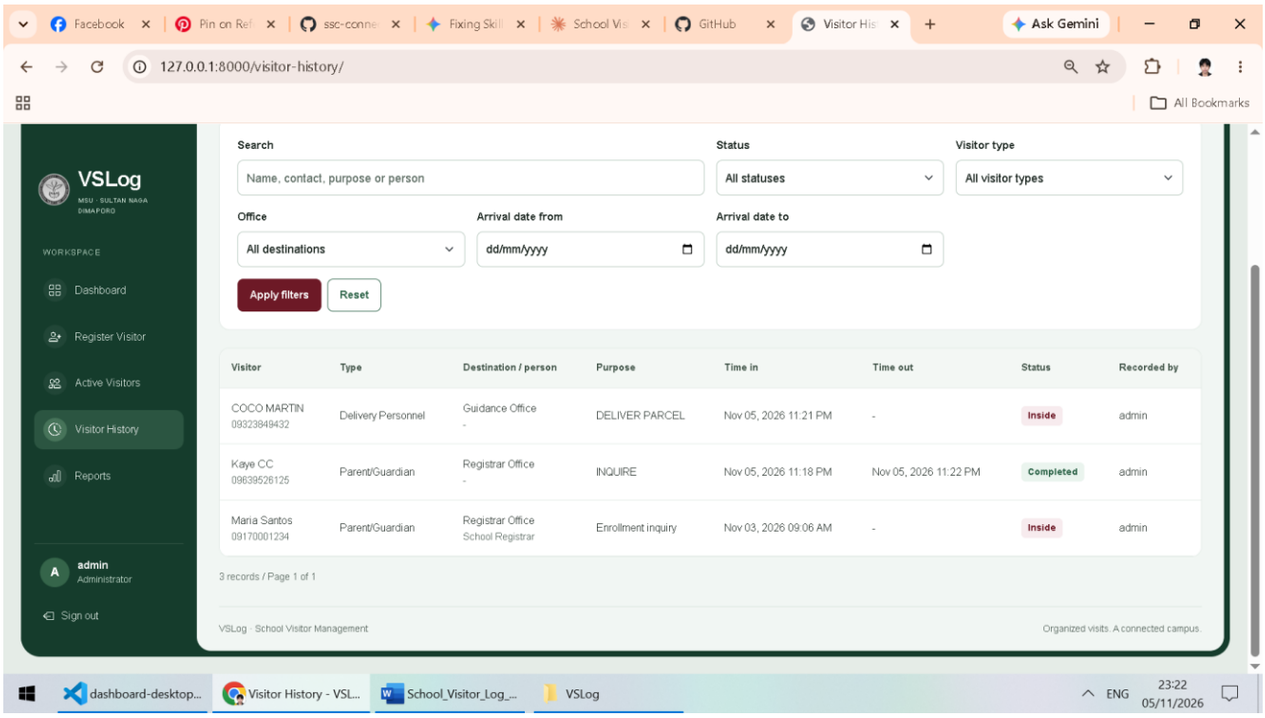


Figure 4. Sample create/read/update/delete operation.

4. Database Implementation

Insert screenshots showing the local MySQL database connection, created tables, and sample records from the practice application.

[INSERT DATABASE SCREENSHOT HERE]

Figure 4. MySQL database and tables.

[INSERT DATABASE SCREENSHOT HERE]

Figure 5. Sample database records.

5. GitHub Repository

Repository URL: [INSERT GITHUB REPOSITORY URL]

Insert screenshots showing the repository, uploaded source code, and commit history.

[INSERT GITHUB SCREENSHOT HERE]

Figure 6. GitHub repository and source files.

[INSERT GITHUB SCREENSHOT HERE]

Figure 7. Git commit history.

6. Learning Reflection

During the technology familiarization activities, the group learned how the different technologies used in web development work together. Python and Django provide the application's backend structure and logic, while HTML5, CSS3, and Bootstrap 5 are used to build a responsive user interface. The group also learned the importance of database design and how Django models can be connected to MySQL for persistent data storage. Git and GitHub introduced the group to version control, allowing changes to be tracked and source code to be stored in a remote repository.

The group may encounter challenges while installing dependencies, configuring the Django project, connecting Django to MySQL, applying database migrations, and using Git commands. These activities help the group understand how to troubleshoot configuration errors, review error messages, verify database settings, and test each component before proceeding to full system development. This section should be revised after the actual activities to include the group's specific experiences.

PART II. REQUIREMENTS AND SYSTEM DESIGN

1. Project Overview

Project Title
Web-Based School Visitor Log Management System

Background and Description
Schools regularly receive visitors such as parents and guardians, alumni, delivery personnel, contractors, government personnel, invited guests, and other individuals. A traditional paper visitor logbook requires visitors or security personnel to write information manually. As records accumulate, locating a previous visit, identifying who is currently inside the school, and preparing visitor summaries can become inconvenient.

The proposed Web-Based School Visitor Log Management System will digitize the recording and monitoring of school visitors. Authorized school personnel will register visitor information, record the purpose and destination of a visit, monitor time-in and time-out, view visitors who are currently inside the premises, search previous records, and view basic visitor statistics. Visitor and visit information will be stored in a MySQL database through a Django-based web application with a responsive Bootstrap interface.

2. Problem Statement
The proposed system intends to address the following problems:

1. Manual visitor logbooks may contain incomplete, inconsistent, or difficult-to-read entries.
2. Searching for a specific previous visitor record can be time-consuming when records are stored only on paper.
3. Authorized personnel may have difficulty quickly determining which visitors are currently inside the school premises.
4. Paper visitor records may be misplaced, damaged, or difficult to organize over time.
5. Recording and checking visitor time-in and time-out manually can make visit monitoring less efficient.
6. Preparing basic visitor counts and summaries requires manual checking and counting of logbook entries.

3. Objectives

General Objective
To design and develop a web-based School Visitor Log Management System that provides an organized and efficient method of recording, monitoring, storing, and retrieving school visitor information.

Specific Objectives
7. To provide authorized school personnel with a digital method for registering visitor and visit information.
8. To record visitor time-in and time-out and maintain the current status of each visit.
9. To display visitors who are currently inside the school premises.
10. To provide searchable and filterable historical visitor records.
11. To store visitor, visit, user, and school-office information in a centralized MySQL database.
12. To provide basic visitor statistics and reports for authorized users.
13. To provide a responsive web interface that can be used on common desktop and mobile screen sizes.

4. Scope and Limitations

Scope
The system will focus on the digital recording and monitoring of non-student visitors entering and leaving the school premises. It will provide authentication for authorized users; visitor registration; visit recording; time-in and time-out management; active-visitor monitoring; visitor history; search and

filtering; basic dashboard statistics; management of authorized user accounts; and management of school offices or destinations. The system will use Django, MySQL, HTML5, CSS3, and Bootstrap 5.

Limitations

The system will not replace student attendance, employee attendance, enrollment, grading, or physical access-control systems. Visitors will not be required to create personal system accounts. The initial version will not include biometric identification, facial recognition, RFID/NFC gates, automated government-ID verification, or automated background checking. The accuracy of visitor records will depend on the information provided by the visitor and entered by authorized personnel.

5. Users and Roles

User	Responsibilities
Administrator	Manages authorized user accounts and school offices/destinations; views all visitor records, active visits, history, and basic reports.
Security Personnel / Authorized Staff	Registers visitors, records visit information and time-in/time-out, views active visitors, and searches visitor history.
Visitor	Provides the information required for visitor registration. The visitor does not require a system account.

6. Functional Requirements

ID	Function	Requirement
FR-01	User Authentication	The system shall allow authorized administrators and staff to log in and log out.
FR-02	Visitor Registration	The system shall allow authorized staff to create and maintain visitor information.
FR-03	Visit and Time-In Recording	The system shall record the purpose, destination, person to visit, date, and time-in of a visitor.
FR-04	Visitor Time-Out	The system shall allow authorized staff to record the visitor's departure time and mark the visit as completed.
FR-05	Active Visitor Monitoring	The system shall display visitors whose visits are currently active or whose status is Inside.
FR-06	Visitor History	The system shall allow authorized users to view previous visit records.
FR-07	Search and Filtering	The system shall support searching and filtering records by relevant fields such as visitor name, date, destination, visitor type, or status.
FR-08	Dashboard and Basic Reports	The system shall display useful counts such as visitors today, currently inside, completed visits, and other basic summaries.

7. Non-Functional Requirements

Category	Requirement
Usability	The interface should use clear labels, consistent navigation, readable forms, and understandable validation/error messages.
Responsiveness	Pages should adapt to common desktop, tablet, and mobile screen sizes using Bootstrap 5.
Security	Restricted functions and visitor records should be accessible only to authenticated and authorized users. Passwords should be handled using Django's authentication mechanisms.

Performance	Common operations such as loading the dashboard, saving a visit, and searching visitor records should respond within a reasonable time under normal school use.
Data Persistence	Saved records should remain available in the MySQL database after the application is restarted.
Maintainability	The project should use organized Django apps, models, templates, URLs, and reusable interface components.

8. Use Case Diagram

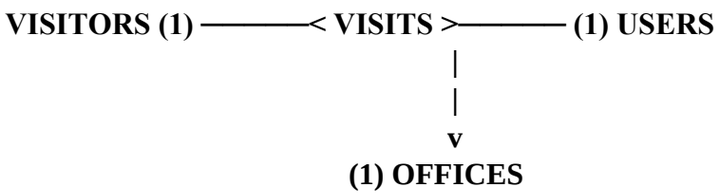
The following specification can be used to draw the formal use case diagram. The Visitor is an external participant who provides information but does not log in.

Actor	Use Cases
Administrator	Log in/out; view dashboard; manage user accounts; manage school offices; view active visitors; view/search visitor history; view basic reports.
Security Personnel / Authorized Staff	Log in/out; register visitor; record visit/time-in; record time-out; view active visitors; search/view visitor history.
Visitor	Provide visitor and visit information to authorized personnel.

[INSERT FORMAL USE CASE DIAGRAM HERE]

9. Entity Relationship Diagram (ERD)

The proposed database contains four principal entities: Users, Visitors, Offices, and Visits. One visitor may have many visit records. One authorized user may record many visits. One school office may be the destination of many visits.



[INSERT FORMAL ERD HERE]

10. Database Table Design

Table	Attribute	Data Type	Key	Description
users	id	BIGINT	PK	Unique user identifier
users	username	VARCHAR(150)	Unique	Login username
users	password	VARCHAR(128)		Django-managed password value
users	first_name	VARCHAR(150)		User first name
users	last_name	VARCHAR(150)		User last name
users	role	VARCHAR(30)		Administrator or Staff role
visitors	visitor_id	BIGINT	PK	Unique visitor identifier
visitors	first_name	VARCHAR(100)		Visitor first name
visitors	last_name	VARCHAR(100)		Visitor last name
visitors	contact_number	VARCHAR(20)		Visitor contact number

visitors	address	VARCHAR(255)		Visitor address
visitors	visitor_type	VARCHAR(50)		Parent/Guardian, Alumni, Delivery, Contractor, Guest, etc.
offices	office_id	BIGINT	PK	Unique office identifier
offices	office_name	VARCHAR(100)		School office/destination name
offices	location	VARCHAR(150)		Office location
visits	visit_id	BIGINT	PK	Unique visit identifier
visits	visitor_id	BIGINT	FK	References visitors
visits	office_id	BIGINT	FK	References offices
visits	recorded_by	BIGINT	FK	References authorized user
visits	purpose	VARCHAR(255)		Purpose of visit
visits	person_to_visit	VARCHAR(150)	Nullable	Person the visitor intends to meet
visits	time_in	DATETIME		Arrival date and time
visits	time_out	DATETIME	Nullable	Departure date and time
visits	status	VARCHAR(20)		Inside or Completed

11. UI Wireframes

At least four interface wireframes are required. Five proposed screens are listed below so the complete core workflow is represented.

Proposed Screen	**Main Contents**
Wireframe 1 – Login	Username and password fields, login button, school/system title.
Wireframe 2 – Dashboard	Visitor counts, currently-inside summary, recent visits, and quick Register Visitor action.
Wireframe 3 – Register Visitor	Visitor details, visitor type, purpose, destination, person to visit, and Time In/Register button.
Wireframe 4 – Active Visitors	Search field and table of visitors currently inside with a Time Out action.
Wireframe 5 – Visitor History	Search/filter controls and a table showing date, visitor, destination, time-in, time-out, and status.

[INSERT WIREFRAME 1 HERE]

[INSERT WIREFRAME 2 HERE]

[INSERT WIREFRAME 3 HERE]

[INSERT WIREFRAME 4 HERE]

[INSERT WIREFRAME 5 HERE]

12. System Workflow

The basic visitor workflow begins when a visitor arrives at the school. Authorized personnel register or locate the visitor, record the visit information, and save the time-in. While the visit is active, the visitor appears in the active-visitors list. When the visitor leaves, authorized personnel records the time-out, the visit is marked as completed, and the record remains available in visitor history and reports.

Visitor Arrives
↓
Security/Staff Logs In
↓
Register or Locate Visitor
↓
Enter Purpose and Destination
↓
Validate and Save Visit
↓
Record Time-In / Status = Inside
↓
Monitor as Active Visitor
↓
Visitor Leaves
↓
Record Time-Out / Status = Completed
↓
Store in Visitor History and Reports

[INSERT FORMAL SYSTEM WORKFLOW DIAGRAM HERE]

Submission Checklist

- Complete the group members and instructor information on the cover page.
- Replace all software-version placeholders with the versions actually installed.
- Insert screenshots of the practice Django application.
- Insert MySQL table and sample-record screenshots.
- Add the GitHub repository URL and screenshots of source code and commits.
- Revise the learning reflection to include the group's actual experiences and errors encountered.
- Insert the formal Use Case Diagram.
- Insert the formal ERD.
- Insert at least four UI wireframes (five are proposed in this document).
- Insert the formal System Workflow diagram.
- Proofread and print the completed hardcopy before the November 6, 2026 morning deadline.